

DM-IDS—A Network Intrusion Detection Method Based on Dual-Modal Fusion

Chao Zha^{1b}, Zhiyu Wang^{1b}, Yifei Fan, Bing Bai^{1b}, Yinjie Zhang^{1b}, Sainan Shi^{1b}, and Ruyun Zhang^{1b}

Abstract—The machine learning-based approach to network intrusion detection presents a groundbreaking research paradigm, positioned to replace traditional rule-based and signature-based methods. However, prior research methodologies have predominantly focused on flow-based approaches, which may not be effective in detecting all types of attacks at a granular level. In this study, we introduce DM-IDS, an attention-convolution architecture model for bimodal network intrusion detection in both flow and payload modalities, using bilinear fusion. Notably, we present a novel method for constructing binary-form feature vectors under the payload modality, with the goal of extracting additional security semantic features. To facilitate this, we independently develop a feature generation tool named Beeman. Finally, we conduct a series of comparative and ablation experiments on two publicly available datasets, CICIDS-2017 and CICIoT-2023, achieving state-of-the-art model performance.

Index Terms—Intrusion detection, flow modal, payload modal, bilinear fusion, semantic.

I. INTRODUCTION

THE METHODOLOGIES employed in network intrusion are constantly evolving, characterized by increased sophistication and stealth. As global expansion of the Internet fosters extensive network interconnectivity [1], it facilitates seamless information sharing and collaboration across diverse domains. However, this enhanced connectivity introduces increased exposure to potential intrusions, making networks more susceptible to sophisticated attack vectors. Even a single system vulnerability can trigger cascading effects, affecting

the integrity, confidentiality, and security of interconnected systems. These vulnerabilities not only compromise critical data and services, but also pose substantial risks to broader infrastructures, including those in the industrial, governmental, and financial sectors [2], [3], [4], [5], [6].

Traditional intrusion detection approaches include signature detection, statistical analysis, and rule-based packet inspection [7]. Signature detection identifies specific patterns or signatures of known attacks created by cybersecurity experts. Statistical analysis uses statistical methods to analyze network traffic, packet frequencies, system resource usage, and other parameters to detect patterns inconsistent with statistical models and reveal abnormal behavior. Rule-based packet inspection uses known attack patterns or abnormal behavior to create rule sets [8]. However, traditional methods require the expertise of professionals, which could require the participation of security experts to provide the prior knowledge required. Furthermore, these approaches exhibit limited adaptability to dynamic network environments and demand continual updates of models or rules, presenting notable drawbacks [9].

With the emergence of machine learning and deep learning, modern intrusion detection systems are favoring methods based on these technologies [10], [11], [12], [13], [14]. These approaches typically involve collecting various types of data such as network traffic, system logs, and event records, incorporating datasets with instances of normal activity and known attacks. Following data collection, relevant features are carefully selected, and machine learning or neural network models are chosen on the basis of the problem's nature and data characteristics. Finally, well-trained models are deployed for proactive monitoring and detection of network data [15].

Current state-of-the-art methods typically utilize flow features [16], [17], [18], where a flow consists of a series of packets that share the same five-tuple information (source IP, destination IP, source port, destination port, protocol value). These features, derived from flow information, such as packet length and transmission rate, have contributed to the success of network intrusion detection systems. However, these methods might struggle with certain types of attack due to the potential lack of prominent flow features, which impacts model training.

Research using payload features for intrusion detection has also yielded promising results [15], [19], [20], [21], [22]. However, certain types of attacks exhibit minimal distinguishable patterns in their payload content, making it challenging to rely solely on payload features for effective differentiation. Consequently, recent studies have begun to

Received 13 December 2024; revised 15 April 2025; accepted 26 April 2025. Date of publication 29 April 2025; date of current version 7 August 2025. This work is supported by the Key Research and Development Program of Zhejiang Province (No. 2024SSYS0001). The associate editor coordinating the review of this article and approving it for publication was L. Cui. (Corresponding author: Ruyun Zhang.)

Chao Zha is with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China, also with the Intelligent Computing Infrastructure Innovation Center, Zhejiang Lab, Hangzhou 311500, Zhejiang, China, also with the University of Chinese Academy of Sciences, Beijing 100049, China, and also with the School of Intelligent Science and Technology, Hangzhou Institute for Advanced Study, UCAS, Hangzhou 311500, Zhejiang, China (e-mail: zhachao21@mailsucas.ac.cn).

Zhiyu Wang, Yifei Fan, Bing Bai, Yinjie Zhang, and Ruyun Zhang are with the Intelligent Computing Infrastructure Innovation Center, Zhejiang Lab, Hangzhou 311500, Zhejiang, China (e-mail: wangzhy@zhejianglab.org; yffan@zhejianglab.org; baibing@zhejianglab.org; zyzj19961126@zhejianglab.org; zcor2021@gmail.com).

Sainan Shi is with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China, and also with the Intelligent Computing Infrastructure Innovation Center, Zhejiang Lab, Hangzhou 311500, Zhejiang, China (e-mail: shisainan22@mailsucas.ac.cn).

Digital Object Identifier 10.1109/TNSM.2025.3565614

TABLE I
COMPARISON OF RECENT INTRUSION DETECTION TECHNIQUES

Methods	Used Technology	Flow Modal	Packet Modal	Low FPR	Fine-Grained	Zero-day
Traditional IDS [30], [31]	predefined rule sets	×	✓	×	✓	×
Wu et al. [32]	GAN, CNN	✓	×	×	✓	×
Shone et al. [33]	NDAE	✓	×	✓	✓	×
Verkerken et al. [34]	AE, OC-SVM, RF, NN	✓	×	✓	✓	✓
Duan et al. [35]	DLGNN	✓	×	×	×	×
Shafiq et al. [36]	CorrAUC, Feature Selection	✓	×	✓	✓	×
Wang et al. [27]	SCAE, SVM	✓	×	×	✓	×
King et al. [12]	MPNN, GAN	✓	×	×	×	×
Yang et al. [16]	CVAE	✓	×	✓	✓	✓
Yang et al. [17]	Tree-Based, K-Means	✓	×	✓	✓	✓
Fu et al. [19]	DFT, Cluster	×	✓	×	✓	✓
Holland et al. [15]	Binary, Auto-ML	×	✓	×	✓	×
Marin et al. [20]	CNN, LSTM, FC	×	✓	×	✓	×
Zhang et al. [21]	LeNet-5, LSTM	×	✓	×	✓	×
Wang et al. [22]	CNN, GRU	×	✓	✓	✓	×
Farrukh et al. [23]	GNN, LLM	✓	✓	×	✓	×
Kiflay et al. [24]	RF, Vote-Based	✓	✓	✓	✓	×
Min et al. [25]	Text-CNN, RF	✓	✓	✓	✓	×
Premkumar et al. [26]	GNN, ML	✓	✓	✓	×	×
Ours	CNN, Attention, Fusion	✓	✓	✓	✓	✓

explore the integration of flow and payload modalities to complement each other's shortcomings [23], [24], [25], [26]. Existing methods primarily employ early-fusion or late-fusion strategies for this integration. Early Fusion combines features at an initial stage, but this approach often introduces additional noise and does not fully exploit the complementary nature of the two modalities [23], [25], [26]. On the other hand, Late Fusion, which focuses on decision-level integration, disregards the critical interdependencies between flow and payload data, limiting its ability to capture their interactions effectively [24].

To address above challenges, we introduce DM-IDS, a dual-modal approach employing flow and payload features through bilinear fusion. Initially, like traditional methods [16], [17], [27], we formulate a *FlowNet* with an attention mechanism as the core module for extraction of flow features. Subsequently, diverging from traditional approaches, we devise a *PayloadNet*, using convolution as the primary module for the extraction of payload characteristics. We argue that encoding payload content in binary form enhances its semantic representation, albeit confronting challenges of significant dimensionality expansion. Convolution helps to mitigate this issue by reducing dimensionality. Ultimately, leveraging the trained *FlowNet* and *PayloadNet*, we perform bilinear fusion on the extracted features for intrusion detection classification.

In the experimental section, we performed comprehensive experiments on two public datasets, CICIDS-2017 [28] and CICIOT-2023 [29]. First, we analyzed our intrusion detection performance, focusing on known attack detection, which demonstrated significant effectiveness. Secondly, we evaluated the impact of payload features by comparing the detection performance of the bilinear fusion method with the single-modal flow feature method. Subsequently, we compared the performance of different fusion methods and selected the bilinear fusion method. Finally, we performed two sets of experiments on 0-day attacks, achieving outstanding performance. Furthermore, we compared our

method with three state-of-the-art methods, demonstrating significant progress.

The primary contributions of this paper are summarized as follows:

1) We present a dual-modal feature generation tool implemented in C++, capable of producing feature vectors for both flow and payload modalities. Furthermore, in the payload modality, we suggested employing binary representation to extract payload content, anticipating improved model performance.

2) We propose a dual-branch Attention-CNN framework for extracting and fusing flow and payload features in network traffic, which better reflects the distinct modality-shared and modality-specific features.

3) We conduct comprehensive experiments on two public datasets, CICIDS-2017 [28] and CICIOT-2023 [29], which included performance analysis and a series of ablation experiments. These experiments validated the state-of-the-art performance of our network intrusion detection and the rationality of our framework design.

The remainder of the paper is organized as follows. Section II introduces related work on current intrusion detection. Section III presents the representation of the data set and the process of extracting features from two different modalities. Section IV shows the detailed designs of our method. In Section V, we experimentally evaluate the performance. Section VI discusses some experimental results and methods presented in this paper. Section VII presents some future work. Finally, we conclude this paper in Section VIII.

II. RELATED WORK

In this section, we summarize related work on AI-based intrusion detection systems (IDS) across different modalities, comparing existing studies to highlight current limitations. Unlike traditional IDS [30], [31], AI-based systems show

improved performance in zero-day detection; however, certain challenges remain, as summarized in Table I.

Flow-Based Methods. Most contemporary network intrusion detection methods are based on flow features. For example, Wu et al. [32] introduced a network intrusion detection algorithm that combines fuzzy rough sets, generative adversarial networks (GAN), and convolutional neural networks (CNN) designed to extract big data in edge networks. Shone et al. [33] proposed a network intrusion detection method grounded in unsupervised feature learning using a symmetric deep autoencoder (NDAE). Verkerken et al. [34] presented a multistage hierarchical intrusion detection approach. The first stage filters out malicious samples using an autoencoder (AE) and a one-class support vector machine (OC-SVM), the second stage then employs a random forest (RF) and a neural network (NN) to classify the samples identified as malicious in the first stage into known attack categories, and the third stage repurposes the first stage's anomaly scores to mitigate false positives. Duan et al. [35] introduced a semi-supervised learning intrusion detection method based on dynamic linear graph neural networks (DLGNN). Shafiq et al. [36] devised a method for feature filtering using the Area Under the Curve (AUC) metric to select features for the chosen ML algorithm, thereby enhancing detection performance. Wang et al. [27] proposed a cloud intrusion detection system based on Stacked Compressed Autoencoders (SCAE) and the Support Vector Machine classification algorithm. King et al. [12] introduced an origin graph analysis method for end-to-end anomaly-based intrusion detection system. Yang et al. [16] proposed a two-stage network intrusion detection method, where the first stage seeks a score measure to minimize the risk of empirically misclassifying known attacks, and the second stage finds another score measure to minimize the risk of inferring unknown attacks. Yang et al. [17] proposed a multilayer hybrid IDS, combining signature-based IDS and anomaly-based IDS to detect both known and unknown attacks in vehicular networks.

Packet-Based Methods. Research on network intrusion detection based on packet features is less extensive compared to that focused on flow features. Notable studies in this domain include Fu et al. [19], who introduced a machine learning-based real-time malicious traffic detection system. This system employs frequency domain feature representation of sequential information to achieve bounded information loss, enhancing detection accuracy while constraining feature dimensionality and improving detection throughput. Holland et al. [15] proposed an automated traffic analysis approach, making machine learning techniques more accessible for a wider range of traffic analysis tasks. Marin et al. [20] explored the capabilities of deep learning models in addressing specific issues related to the detection and classification of malicious software network flows. They directly considered raw measurement values from monitored byte streams as input to evaluate differences in various representations of raw traffic features proposed by the model. Zhang et al. [21] integrated an improved structure of the LeNet-5 and LSTM neural networks, simultaneously learning the spatial and temporal features of flows. Wang et al. [22] introduced a deep hierarchical network

for the detection of malicious traffic at packet level based on deep learning, capable of learning traffic features from raw packet data.

Dual-Modal Methods. In recent years, researchers have begun to explore the use of dual-modal data for intrusion detection. For example, Farrukh et al. [23] proposed a framework that combines flow-level and packet-level data into a heterogeneous graph structure, using a heterogeneous graph neural network (HGNN) for graph-level classification to analyze network traffic and intrusion detection. This framework also integrates large language models (LLMs) to generate detailed, human-readable explanations and recommended remediation measures. Kiflay et al. [24] introduced an approach in which two random forest models separate the flow-based features and payloads of the protocol, and the final classification of network traffic is achieved by aggregating the predictions of both models using a soft voting method. Min et al. [25] suggested transforming the payloads of network traffic into continuous vector representations using word embedding techniques, then extracting features using Text-CNN, and finally combining these with flow statistics for classification using a random forest model. Premkumar et al. [26] developed a context-sensitive network intrusion detection system based on graph representation learning (GRL), modeling entities and their relationships in network traffic using graph structures. Using the CIC-IDS2017 dataset, they explored two methods to represent NIDS data at the flow and packet levels and employed graph neural networks and graph embedding algorithms to create low-dimensional vector representations as a complement to traditional network features, thus developing an effective intrusion detection approach.

Table I provides a summary of related work in five key points: data modality, false positive rate (specifically, the rate of benign misclassified as attack), fine-grained classification and zero-day attack detection. Traditional anomaly-based or signature-based methods perform well in recognizing known attacks and offer good interpretability, yet they often struggle with unknown attacks and tend to exhibit high false positive rates [30], [31]. Flow-based intrusion detection methods have achieved significant success in recent studies, although some suffer from high false positive rates [12], [27], [32], [35], while others are similarly ineffective against unknown attacks [12], [27], [32], [33], [35], [36]. Only three studies have managed to balance low false positive rates, fine-grained detection, and zero-day detection simultaneously [16], [17], [34]. However, we observed that flow-based methods underperform in classifying certain types of attacks because their distinguishing characteristics lie in payload data, where flow features might resemble benign traffic. Packet-based intrusion detection methods have received comparatively less attention, mainly because flow feature extraction is simpler and better suited to real-time scenarios. Existing packet-based approaches generally show average performance in terms of false positive rates and zero-day detection, which we believe is due to challenges similar to those in traditional anomaly-based and signature-based methods [30], [31]. Specifically, the inability to recognize unfamiliar attack patterns and the limited transferability of

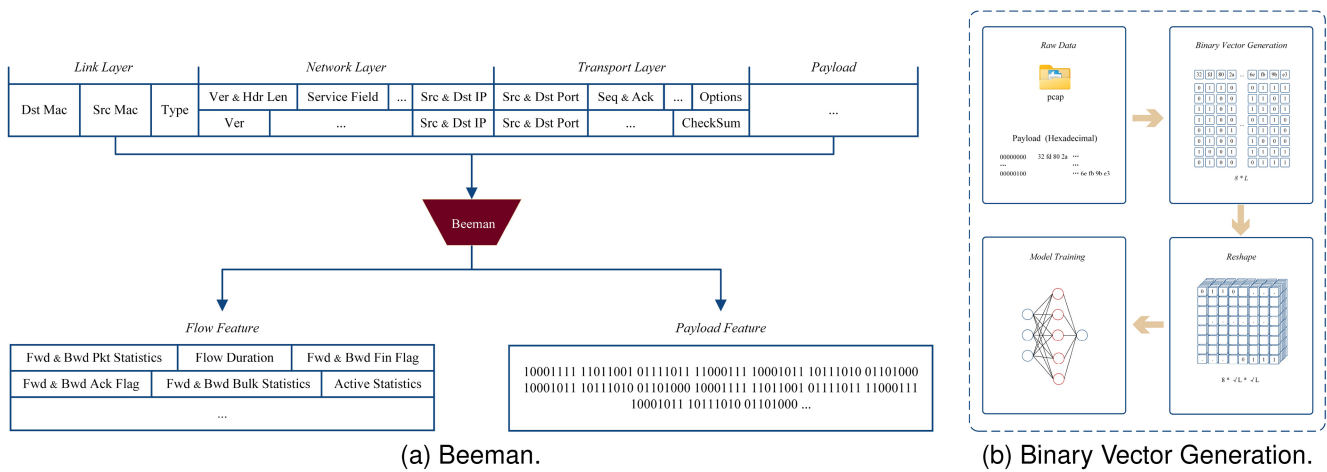


Fig. 1. Beeman introduces the process of generating two major modal features based on the content of network data, including the flow feature modality and the payload feature modality. (a) Beeman. (b) Binary Vector Generation.

payload features are significant limitations for these methods in detecting unknown attacks.

Methods combining flow and packet modalities have gained attention in recent years [23], [24], [25], [26], overcoming some of the limitations outlined above. However, we identify several remaining challenges. First, when using payload data as features, these methods often rely on natural language processing techniques, such as word embedding. However, domain-specific data, such as code, encoded, or encrypted content, differ from natural language, as observed in certain studies [37]. Second, in terms of feature fusion strategies, we categorize fusion methods into three types: Early, Intermediate, and Late Fusion. For example, [23], [25], [26] employs Early Fusion by integrating raw features directly, while [24] utilizes Late Fusion, making decisions via a voting mechanism.

We advocate for Intermediate Fusion, as it allows each modality to retain an independent processing flow, allowing features in each modality to be learned separately without forcing early integration. Compared to Early Fusion, Intermediate Fusion can capture higher-level features from each modality via deep networks before merging, thereby reducing noise and enhancing critical information. In addition, intermediate fusion incorporates intermodal interactions in the intermediate layers of the model, strengthening the model's ability to capture multimodal associations. This approach also offers greater adaptability and transferability. If data from one modality are unavailable, the model can still achieve comprehensive feature fusion and prediction based on the remaining modality. Its improved transferability makes Intermediate Fusion particularly effective in detecting unknown attacks.

III. DATASETS

In the realm of deep learning and machine learning research [38], [39], the appropriate representation of data is of paramount importance in improving model performance. Nonetheless, within the domain of cybersecurity, the treatment of data representation poses a distinctive challenge. At times, the adoption of processing methods identical to those employed in some fields, such as natural language

processing and image processing, proves impractical. Instead, an imperative arises to amalgamate techniques specifically attuned to the nuanced context of cybersecurity. Subsequent sections will explain the manifestations of data representation in Section III-A, followed by a detailed exposition of our approach to data preprocessing in Section III-B.

A. Data Representation

Network packets constitute the fundamental entities transmitted within computer networks, encapsulating information originating from the source host and destined for the target host [40]. In this context, a succinct examination of the packet format will be presented, using it as a foundational elucidation for our proposed data representation approach. The structure of network packets is conventionally contingent upon the specific protocol in use, yet it can be generally depicted, as illustrated in Fig. 1(a).

Primarily, data packets encapsulate attributes pertaining to the data link layer, the network layer, and the transport layer. These attributes include crucial information, such as source and destination MAC addresses, IP addresses, and ports. Furthermore, these packets integrate distinctive fields that capture key aspects of the network environment, including but not limited to window size, SYN, ACK, FIN, and RST flags, among others. Second, data packets include payload content, commonly denoting information from the presentation layer. This may manifest itself as the request or response content in protocols such as HTTP.

The utilization of pertinent fields from the data link layer, network layer, and transport layer, specifically IP addresses and port numbers, is not deemed an appropriate method for feature extraction in our approach. Instead, we advocate for a more favorable strategy centered around these fields. This approach aligns with established research methodologies, in which a key is established for a flow based on source IP, destination IP, source port, destination port, and the transport layer protocol value. Subsequently, various statistical analyses are applied to this flow, encompassing parameters such as forward and backward packet length, forward and backward

packet byte counts, flow duration, and statistics pertaining to network environment identification fields. In this context, our methodology is influenced by previous studies that have demonstrated the significant impact of such data on the effectiveness of intrusion detection [16], [17], [27], [32]. We intend to integrate these features as a modality input for training and prediction within our model.

Moreover, our objective is to harness the payload content. The payload content typically lacks structured characteristics, primarily comprising information intended for transmission. Although from a certain perspective, it may bear resemblance to natural language, it inherently harbors elements of potentially malicious content. This content not only encapsulates semantics expressible in natural language but also involves cybersecurity semantics. In this regard, we refrain from employing methodologies akin to those utilized in natural language processing for segmenting the payload content into features. Instead, we opt to transform the payload content into binary form, considering binary as the fundamental language at the lowest computational level. Our aspiration is for the model to acquire additional semantic knowledge built upon this binary representation. This innovative approach will function as an alternative modality input for both the training and prediction phases within our model.

In conclusion, our final data representation methodology will incorporate two modalities: the flow feature modality and the packet payload feature modality. We envisage undertaking feature fusion to augment the effectiveness of intrusion detection.

B. Pre-Processing

We have developed a feature generation tool in C++ (g++ 9.4.0) that generates dual-modal feature vectors as inputs for our model, namely **Beeman**. These vectors encompass both flow feature vectors and payload feature vectors. Utilizing a multi-threaded architecture, we conducted packet content parsing and computed diverse flow-related features, including statistics on forward and backward packet lengths, counts for both directions, and TCP flag field statistics, among others. The statistical analysis employed in this process was referenced from CICFlowMeter [28].

Furthermore, the payload content underwent transformation into binary vectors and was subsequently associated with the corresponding flows. The length of payload content is typically indeterminate. In this context, we have imposed specific constraints, such as truncating the payload to the initial 128 or 256 bytes, for the generation of payload feature vectors. The vector length remains adjustable.

The Fig. 1(b) illustrates the process of converting hexadecimal payload content into a binary vector. Firstly, each byte content is converted into an 8-bit binary representation. Then, it is represented as 8 channels, with each binary bit corresponding to one channel. Therefore, assuming an L -byte hexadecimal payload content will be converted into an $L * 8$ channel binary vector representation. Finally, it is reshaped as needed for training purposes.

The multithreaded architecture significantly boosts the efficiency of our data processing, offering the flexibility to configure the number of threads based on available computational resources. The code for this component has been compiled into a dynamic library, streamlining the deployment of model training and prediction in collaboration with the Python language in subsequent stages.

Following the generation of the aforementioned dual-modal feature vectors, we conducted min-max normalization on the flow feature vectors. This process aims to alleviate the influence of disparate scales among features, expedite model convergence speed, and enhance overall model stability. The normalization procedure is delineated in Eq. (1):

$$X_n = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (1)$$

Here, X is the original data, X_n is the normalized data, $X_{\min}$ and $X_{\max}$ represent the minimum and maximum values of the feature in the original dataset, respectively.

IV. METHODOLOGY

In this section, we present the methodology of **DM-IDS**, i.e., the overall architecture and the design of three main modules in **DM-IDS**.

A. Overall Architecture

Fig. 2 depicts the training process and the dual modal fusion process proposed in this paper for network intrusion detection problems.

Our method performs detail fusion on two primary inputs: **Flow Net** (FNet) and **Payload Net** (PNet). The comprehensive training procedure is delineated as follows:

- **Feature Generation and Normalization:** Initially, we used our developed tool, **Beeman**, for both feature generation and data normalization. This encompassed the creation of dual-modal feature vectors, specifically the flow feature vector V_{fi} and the payload feature vector V_{pi} .
- **Feature Extraction:** Subsequently, the dual-modal feature vectors were individually input into the **Flow Net** and **Payload Net** to execute feature encoding, resulting in feature vectors V_{fo} and V_{po} .
- **Feature Fusion:** Following the encoding process, we applied **Bi-Linear Feature Fusion** to vectors V_{fo} and V_{po} .
- **Classification with Fully Connected Classifier:** Ultimately, the fused feature vector was classified through a classifier that included fully connected layers. In this stage, we utilized the **Cross-Entropy Loss Function** to minimize the classification error, defined as follows:

$$CrossEntropyLoss = - \sum_{i=1}^C y_i \cdot \log(p_i) \quad (2)$$

Here, assuming there are C categories, y_i represents the true class of the sample (a one-hot encoded vector indicating that the sample belongs to class i), and p_i denotes the model's predicted probability for class i .

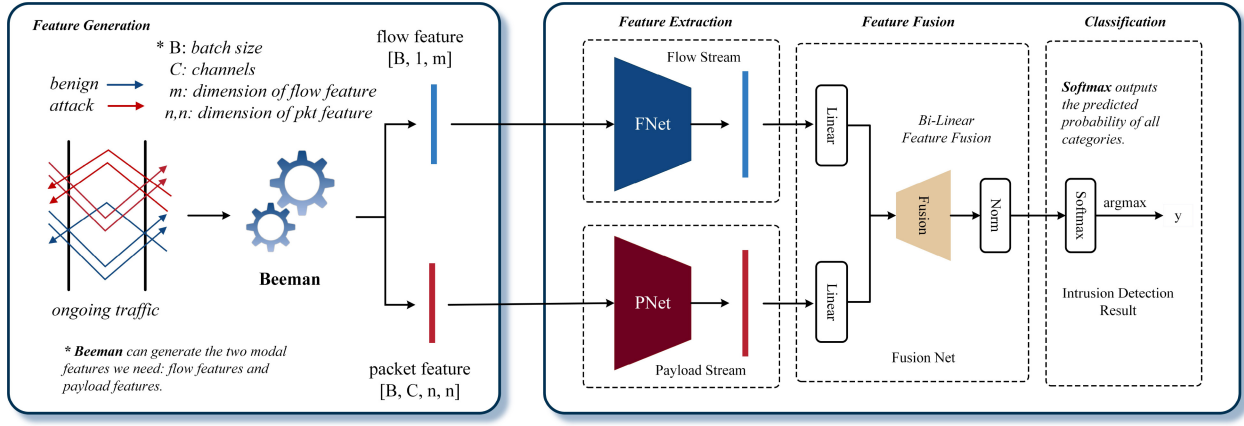


Fig. 2. Overall Architecture of Our Proposed Methods (The **Feature Generation** module harnessed Beeman for the creation of bimodal feature vectors. The **Feature Extraction** module, we employed an *Attention-Convolution* architecture to derive salient features from the bimodal feature vectors, with the objective of more effectively delineating nuances among distinct data categories. The **Feature Fusion** stage employed a *Bi-Linear Fusion* method to seamlessly amalgamate the vectors obtained post feature extraction. The **Classification** component is composed of a fully connected classifier).

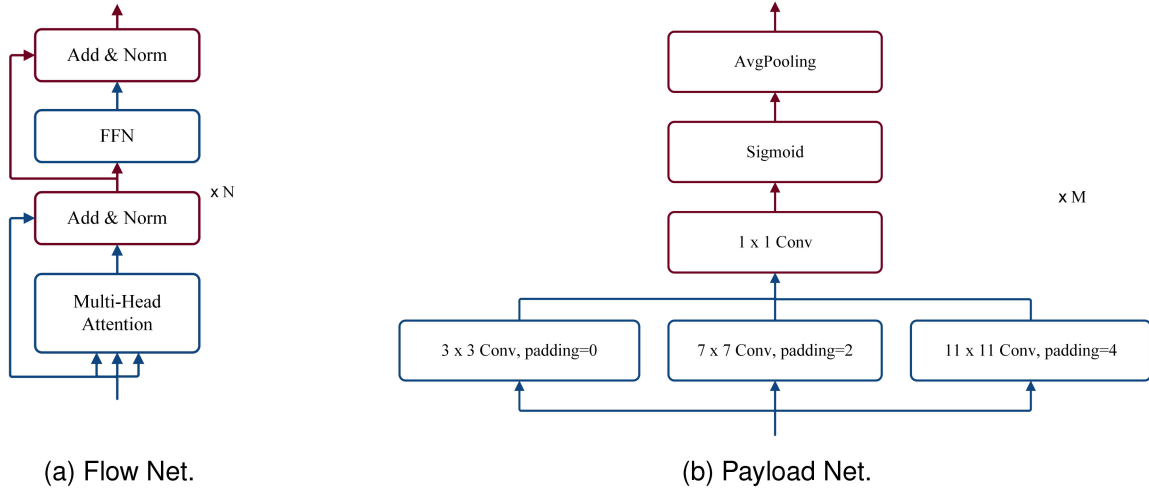


Fig. 3. The Overall Structure of the Feature Extraction (The **Flow Net** consists of N multi-head attention blocks, each comprising a multi-head attention sub-layer and a feed-forward neural network sub-layer. The **Payload Net** is composed of M convolution blocks, each containing multiple parallel convolutional layers with different kernel sizes, a 1×1 convolutional layer, and a pooling layer).

B. Flow Net

The **Flow Net** is composed of identical attention blocks N , as shown in Fig. 3(a), each consisting of two stacked sub-layers: a *Multi-Head Attention Layer* and a *Feed-Forward Neural Network Layer* [41]. The *Multi-Head Attention Layer* encompasses a multi-head attention neural network, a residual connection, and layer normalization. Similarly, the *Feed-Forward Neural Network Layer* incorporates a feedforward neural network, a residual connection, and layer normalization.

The input to **Flow Net** is a flow feature vector V_{fi} with a shape of $[B, 1, m]$, where B denotes the batch size, and m represents the number of flow features. After passing through the **Flow Net**, an encoded vector V_{fo} of the same shape is obtained. The attention function can be represented by the following formula:

$$\text{Attention}(V_{fi}, V_{fi}, V_{fi}) = \text{softmax}\left(\frac{V_{fi} \cdot V_{fi}^T}{\sqrt{d_k}}\right) \cdot V_{fi} \quad (3)$$

where d_k represents the dimension of the input vector V_{fi} .

Subsequently, the process of the **Flow Net** can be represented by the following set of equations:

$$F_{attn} = \text{LayerNorm}(V_{fi} + \text{Attention}(V_{fi}, V_{fi}, V_{fi})) \quad (4)$$

$$V_{fo} = \text{LayerNorm}(F_{attn} + \text{FFN}(F_{attn}, F_{attn}, F_{attn})) \quad (5)$$

Here, *LayerNorm* represents the layer normalization operation and *FFN* denotes the neural network that is transmitted.

C. Payload Net

The **Payload Net** consists of M identical *Convolutional Blocks*, as illustrated in Fig. 3(b). Each *Convolutional Block* comprises three parallel convolutional layers with kernel sizes of 3×3 , 7×7 , and 11×11 , and padding values of 0, 2, and 4, respectively [42], [43]. We aim to extract multidimensional features of the payload content through convolutions with different receptive fields. The vectors outputted by these three parallel convolutional layers have the same shape. Following these layers is a 1×1 convolution to facilitate the interaction of information across the three different channels. Subsequently,

a sigmoid activation layer and an average pooling layer are applied to the feature vectors, serving to reduce dimensionality.

The feature vectors of the payload modality, derived from binary encoding, frequently confront challenges related to excessive dimensionality, prompting the need for dimensionality reduction. Convolutional operations are effective in addressing this issue. Furthermore, the utilization of various convolutional kernel sizes to define different receptive fields is used to extract multidimensional features. This approach ensures that our model can capture a more extensive range of information.

The input to the **Payload Net** is a payload feature vector V_{pi} with a shape of $[B, C, n, n]$, where B is the batch size, C is the number of channels and $[n, n]$ represents the shape of an individual feature vector. Three parallel convolutional process can be expressed using the following formula:

$$A_{pi}^i = \text{Conv}\left(V_{pi}^i \odot W_{(3,3)}^i, \beta_A^i\right) \quad (6)$$

$$B_{pi}^i = \text{Conv}\left(V_{pi}^i \odot W_{(7,7)}^i, \beta_B^i\right) \quad (7)$$

$$C_{pi}^i = \text{Conv}\left(V_{pi}^i \odot W_{(11,11)}^i, \beta_C^i\right) \quad (8)$$

Here, Conv denotes a convolutional layer with parameters W and β , i represents the i -th channel, $\odot$ denotes the element-wise multiplication, where $(3, 3)$, $(7, 7)$, and $(11, 11)$, respectively, indicate the shapes of the corresponding convolutional kernel parameter matrices.

Following this, a 1×1 convolution is applied to the outputs A_{pi}^i , B_{pi}^i , and C_{pi}^i of three parallel convolutional layers. The process can be represented as follows:

$$V_{po} = \text{Cat}\left(A_{pi}^i, B_{pi}^i, C_{pi}^i\right) \quad (9)$$

where i represents the i -th channel, Cat denotes the 1×1 convolution.

D. Feature Fusion

After the feature extraction from two distinct modalities using separate encoders, two feature vectors, V_{po} and V_{fo} , of disparate dimensions are generated, V_{po} and V_{fo} . In this paper, we introduce a **Bi-Linear Fusion Method** for modality integration [44], this part of the work is illustrated in Fig. 2. Specifically, V_{po} and V_{fo} undergo individual processing through *Fully-Connected Layers* to produce feature vectors of uniform dimensionality. Subsequently, the *Element-wise Multiplication* of the resulting vectors produces a fused vector V_{fusion} , followed by the application of *Layer Normalization*.

We assume that the dimensions of V_{fo} and V_{po} are $[B, 1, m']$ and $[B, 1, j']$, where B denotes the batch size, m' and j' represent the dimensions of flow features and packet features after feature extraction, respectively. After fusion, the dimension becomes $[B, 1, k]$. The fusion process can be represented by the following set of equations:

$$F_{fo} = \text{FC}\left(V_{fo} \cdot W_{(m,k)}, \beta_{(1,k)}\right) \quad (10)$$

$$F_{po} = \text{FC}\left(V_{po} \cdot W_{(j,k)}, \beta_{(1,k)}\right) \quad (11)$$

$$V_{\text{fusion}} = \text{LayerNorm}(F_{fo} \odot F_{po}) \quad (12)$$

Here, FC denotes a full-connected layer with parameters W and β , where (m, k) , (j, k) , and $(1, k)$, respectively, indicate the shapes of the corresponding parameter matrices, $\odot$ denotes the element-wise multiplication, LayerNorm represents the layer normalization operation.

Moreover, we have explored two fusion methods: a **Concatenation Fusion** approach and an **Attention-Based Fusion** technique [45], [46]. The **Concatenation Fusion** method combines feature vectors from distinct modalities through elementary operations like concatenation and weighted summation. These straightforward operations minimize interparameter dependencies, and subsequent network layers naturally adapt to such operations. Conversely, the **Attention-Based Fusion** method accomplishes fusion by learning correlations between feature vectors from various modalities.

E. Training

In summary, following the aforementioned description, our training procedure includes the following steps: Initially, we parsed the raw data through **Beeman**, yielding dual-modal feature vectors V_{fi} and V_{pi} . Subsequently, V_{fi} and V_{pi} underwent additional feature extraction in **Flow Net** and **Payload Net**, which produced feature vectors V_{fo} and V_{po} , respectively. Subsequently, **bi-linear feature fusion** was applied to V_{fo} and V_{po} , producing a fused feature vector V_{fusion} . Lastly, a classifier consisting of fully connected layers was utilized, with the classification results subjected to minimizing the *Cross-Entropy Loss Function*.

Especially notable is that during our training process, we used the **Adam** optimizer and implemented a **warm-up** strategy for the learning rate. The specific warmup strategy can be expressed by the following formula:

$$lr = \begin{cases} base_lr * \frac{step}{W}, & step \leq W \\ \max\left(0, \frac{train_step - step}{\max(1, train_step - W)}\right), & step > W \end{cases} \quad (13)$$

Here, $base_lr$ represents the initial value of the learning rate, $step$ indicates the current step, $train_step$ represents the total training steps, and W is a predefined threshold parameter.

The specific training procedures for **DM-IDS** are summarized in Algorithm 1.

F. Computational Complexity Analysis

As shown in Algorithm 1, DM-IDS involves three main modules: **FNet**, **PNet**, and **Feature Fusion**. In this section, we will perform a computational complexity analysis of these three core modules.

FNet consists of multiple attention modules, each comprising a multi-head self-attention layer, two residual connections with layer normalization, and a feedforward neural network layer. As described in Section IV-B, the input dimension of FNet is $[B, 1, m]$. Consequently, the algorithmic complexity of the **FNet** in each batch can be detailed as follows:

1) Multi-head self-attention layer.

a) Linear transformation to generate Q , K , and V : $O(B \cdot m^2)$.

b) Calculation of attention weights QK^T : $O(B \cdot m)$.

Algorithm 1 Training DM-IDS**Input:** Raw data and labels Y .**Output:** *Classifier*.

```

1:  $V_{fi}, V_{pi} \leftarrow \text{Beeman}$ .
2:  $V_{fi}, V_{pi} \leftarrow \text{Eq. (1)}$ .
3: for  $ep = 1, 2, \dots, epoch\_num$  do
4:   for  $batch = 1, 2, \dots, batch\_num$  do
5:      $V_{fo} \leftarrow \text{use Eq. (3), Eq. (4) and Eq. (5)}$ .
6:      $V_{po} \leftarrow \text{use Eq. (6), Eq. (7), Eq. (8) and Eq. (9)}$ .
7:      $V_{fusion} \leftarrow \text{use Eq. (10), Eq. (11) and Eq. (12)}$ .
8:      $P_y \leftarrow \text{Classifier}(V_{fusion})$ .
9:      $loss \leftarrow \text{use Eq. (2)}$ .
10:     $lr \leftarrow \text{use Eq. (13)}$ .
11:     $\text{Classifier} \leftarrow \text{Classifier} - lr \cdot \nabla_{\text{Classifier}} \cdot loss$ 
12:  end for
13: end for

```

TABLE II
HARDWARE AND SOFTWARE DEVICE DETAILS OF THE
EXPERIMENTAL SERVER

Device	Device Details
Operating System	Ubuntu 20.04 LTS
Program Language	Python 3.8.12 & C++ (g++ 9.4.0)
Processor	Intel(R) Xeon(R) Gold 6138 CPU @2.00GHz
GPU	NVIDIA A100-PCIE-40GB
Memory	128GB RAW

- c) Softmax operation: $O(B \cdot m)$.
- d) Linear transformation after concatenating multiple heads: $O(B \cdot m^2)$.
- 2) Residual connection and layer normalization.
 - a) Residual connection (addition of input and output): $O(B \cdot m)$.
 - b) Layer normalization (calculation of mean, standard deviation, and normalization): $O(B \cdot m)$.
- 3) Feedforward neural network layer.
 - a) First linear transformation (maps $[B, 1, m]$ to $[B, 1, f]$, where $f = 2m$): $O(B \cdot m^2)$.
 - b) Activation function: $O(B \cdot m)$.
 - c) Second linear transformation (maps $[B, 1, f]$ to $[B, 1, m]$, where $f = 2m$): $O(B \cdot m^2)$.

Accordingly, the overall algorithmic complexity of FNet is $O(B \cdot m^2) + O(B \cdot m) + O(B \cdot m^2)$, which simplifies to $O(B \cdot m^2)$.

PNet consists of three parallel convolutional layers and a 1×1 convolutional layer for concatenation. As described in Section IV-C, the input dimension of PNet is $[B, C, n, n]$. Therefore, the algorithmic complexity of **PNet** for each batch can be expressed as follows:

- 1) Parallel convolutions.
 - a) Convolution layer (3×3 kernel): $O(B \cdot 9 \cdot C \cdot n^2)$.
 - b) Convolution layer (7×7 kernel): $O(B \cdot 49 \cdot C \cdot n^2)$.
 - c) Convolution layer (11×11 kernel): $O(B \cdot 121 \cdot C \cdot n^2)$.
- 2) Concatenation layer (1×1 kernel): $O(B \cdot 3C \cdot n^2)$.

Accordingly, the overall algorithmic complexity of **PNet** is $O(B \cdot 9 \cdot C \cdot n^2) + O(B \cdot 49 \cdot C \cdot n^2) + O(B \cdot 121 \cdot C \cdot n^2) + O(B \cdot 3C \cdot n^2)$, which simplifies to $O(B \cdot C \cdot n^2)$.

TABLE III
FEATURES USED IN OUR EXPERIMENTS

Type	Features
Time-Realtd	Flow Duration, Flow IAT Mean, Flow IAT Std, Flow IAT Max, Flow IAT Min, Fwd IAT Tot, Fwd IAT Mean, Fwd IAT Std, Fwd IAT Max, Fwd IAT Min, Bwd IAT Tot, Bwd IAT Mean, Bwd IAT Std, Bwd IAT Max, Bwd IAT Min, Active Mean, Active Std, Active Max, Active Min, Idle Mean, Idle Std, Idle Max, Idle Min
Packet-Realtd	Tot Fwd Pkts, Tot Bwd Pkts, Flow Bwts/s, Flow Pkts/s, Fwd Pkts/s, Bwd Pkts/s, Subflow Fwd Pkts, Subflow Fwd Bwts, Subflow Bwd Pkts, Subflow Bwd Bwts, Init Fwd Win Bwts, Init Bwd Win Bwts, Fwd Act Data Pkts
Length-Realtd	TotLen Fwd Pkts, TotLen Bwd Pkts, Fwd Pkt Len Max, Fwd Pkt Len Min, Fwd Pkt Len Mean, Fwd Pkt Len Std, Bwd Pkt Len Max, Bwd Pkt Len Min, Bwd Pkt Len Mean, Bwd Pkt Len Std, Fwd Header Len, Bwd Header Len, Pkt Len Min, Pkt Len Max, Pkt Len Mean, Pkt Len Std, Pkt Len Var, Pkt Size Avg, Fwd Seg Size Avg, Bwd Seg Size Avg, Fwd Seg Size Min
Protocol-Related	Fwd PSH Flags, Bwd PSH Flags, Fwd URG Flags, Bwd URG Flags, FIN Flag Count, SYN Flag Count, RST Flag Count, PSH Flag Count, ACK Flag Count, URG Flag Count, CWE Flag Count, ECE Flag Count, Down/Up Ratio

Feature Fusion consists of two fully connected layers, an element-wise multiplication and a layer normalization layer. As described in Section IV-D, the input dimension of **Feature Fusion** is $[B, 1, m']$ and $[B, 1, j']$, and the output dimension of **Feature Fusion** is $[B, 1, k]$. Therefore, the algorithmic complexity of **Feature Fusion** for each batch can be expressed as follows:

- 1) Fully connected layer: $O(B \cdot m' \cdot k) + O(B \cdot j' \cdot k)$.
- 2) Element-wise multiplication: $O(B \cdot k)$.
- 3) Layer normalization: $O(B \cdot k)$.

Accordingly, the overall algorithmic complexity of **Feature Fusion** is $O(B \cdot m' \cdot k) + O(B \cdot j' \cdot k) + O(B \cdot k) + O(B \cdot k)$, which simplifies to $O(B \cdot k^2)$, where $m' \approx j' \approx k$.

V. EXPERIMENTAL DESIGN AND RESULTS

In this section, we elaborate on our experimental findings and undertake a comprehensive analysis to elucidate the strides made in our research endeavors. Using a meticulous experimental methodology, our primary objective is to underscore the efficacy and preeminence of the proposed approach.

A. Experimental Setting

In the pursuit of advancing our proposed network intrusion detection method, we used a combination of the C++ and Python programming languages. Complementing our development efforts, we harnessed well-established libraries, including pandas, numpy, and pytorch. The orchestration of our computational tasks was facilitated by a hardware ensemble comprising 2 NVIDIA A100-PCIE-40GB GPUs, each with detailed specifications meticulously presented in Table II.

TABLE IV
DATASET PARTITION SETTINGS

Datasets	Setting1	Setting2
CICIDS-2017	Benign, FTP-Patator, SSH-Patator, Dos-Slowloris, Dos-SlowHttptest, Dos-Hulk, DDos, Bot	Benign, FTP-Patator, SSH-Patator, Dos-SlowHttptest, Dos-GoldenEye, Brute-Force, Portscan, Bot
CICIoT-2023	Benign, DDoS-Ack-Fragment, DDoS-Http-Flood, DDoS-ICMP-Fragment, Mirai-Greeth-Flood, VulnerabilityScan	Benign, VulnerabilityScan, DDoS-ICMP-Fragment, SqlInjection, DDoS-Ack-Fragment, CommandInjection

B. Datasets

In our experimentation, we opted for the use of two publicly accessible datasets, namely CICIDS-2017 [28] and CICIoT-2023 [29] (details of the features used in the dataset can be found in Table III). The CICIDS-2017 dataset encapsulates benign activities, as well as contemporary and prevalent attacks, faithfully emulating real-world data in the form of PCAPS. Data acquisition for this dataset spans a period of 5 days, encompassing a spectrum of incidents including *Brute-Force FTP*, *Brute-Force SSH*, *Denial of Service (DoS)*, *Web-Based Attacks*, and *Distributed Denial of Service (DDoS)*, among other categories. The CICIoT-2023 dataset, on the other hand, stands out as an innovative repository of Internet of Things (IoT) attack data. This compilation incorporates a diverse array of attacks, including, but not limited to *Distributed Denial of Service (DDoS)*, *Reconnaissance*, *Brute-Force*, *Deceptive Maneuvers*, and instances of notorious *Mirai malware*.

In previous research endeavors, numerous investigations have adopted a proportional random split methodology to segregate datasets into training and testing sets. We posit that such a partitioning strategy may culminate in the convergence of data distributions between training and testing sets, potentially engendering inflated experimental outcomes. Our proposed partitioning approach adheres to the prevalent 7:3 ratio for training and testing sets. However, we introduce a temporal order based on timestamps, allocating the initial 70% of the data for training and the subsequent 30% for testing. Moreover, we have categorized each dataset into two subsets, with specific partitioning intricacies elucidated in Table IV.

C. Evaluation on Public Datasets

In the first stage, we deployed our proposed feature generation tool, **Beeman**, to extract around 80 flow features from two publicly accessible datasets, namely CICIDS-2017 and CICIoT-2023. Additionally, Beeman generated binary payload feature vectors, enforcing a length constraint of 256 bytes for the payload component. To implement the length constraint strategy effectively, we computed length statistics for all packets. Consequently, we conducted tests within this length range to optimize available data for validation, excluding packets that did not meet this criterion.

Furthermore, we excluded features such as five-tuples and timestamps from all data. In this testing environment, we assert that five-tuples and timestamps may exhibit a significant correlation with category labels [47], [48]. However, our proposed method does not require time series features. The details of the

TABLE V
TRAFFIC DETAILS OF TWO PUBLICLY DATASETS

Settings	Label	Train	Test
CICIDS-2017-Setting1	Benign	5947	2549
	FTP-Patator	1995	855
	SSH-Patator	1388	595
	Dos-Slowloris	2436	1044
	Dos-SlowHttptest	2492	1068
	Dos-Hulk	2408	1032
	DDos	2930	1256
	Bot	5054	2166
CICIDS-2017-Setting2	Benign	5215	2235
	FTP-Patator	1999	857
	SSH-Patator	1344	576
	Dos-SlowHttptest	2566	1100
	Dos-GoldenEye	504	216
	Brute-Force	418	179
	Portscan	112	48
	Bot	1708	732
CICIoT-2023-Setting1	Benign	8400	3600
	DDoS-Ack-Fragment	8400	3600
	DDoS-Http-Flood	8400	3600
	DDoS-ICMP-Fragment	8400	3600
	Mirai-Greeth-Flood	8400	3600
	VulnerabilityScan	8400	3600
CICIoT-2023-Setting2	Benign	8400	3600
	SqlInjection	1194	511
	VulnerabilityScan	8400	3600
	DDoS-ICMP-Fragment	8400	3600
	DDoS-Ack-Fragment	8400	3600
	CommandInjection	7000	3000

extracted training set and the data set are precisely delineated in Table V.

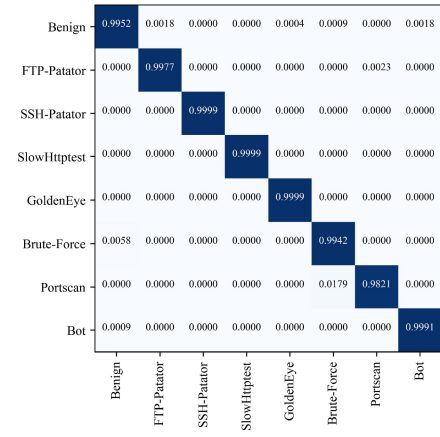
In the second stage, experiments were carried out on two datasets, which is equivalent to four runs (with two settings for each dataset). The classification results are depicted in the confusion matrices presented in Fig. 4, where the horizontal labels denote predicted categories and the vertical labels denote true categories.

The experimental findings underscore the significantly advanced performance of our model. In the Setting 1 of the CICIDS-2017 dataset, we achieved recall rates of nearly 99% or higher across predictions for 8 distinct classes. Similarly, in the Setting 2 of the CICIDS-2017 dataset, we consistently maintained recall rates of 99% or higher in the majority of categories.

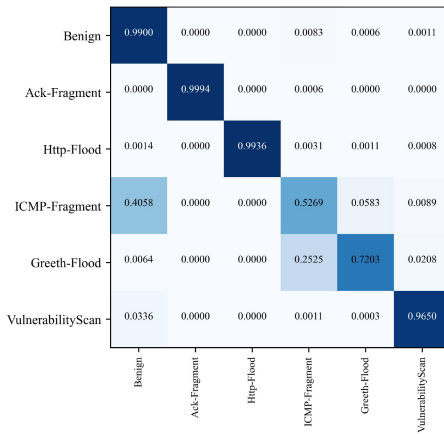
Turning our attention to the CICIoT-2023 dataset Setting 1, we achieved recall rates exceeding 96% in four categories, with the remaining two at 53% and 72%. Similarly, in the CICIoT-2023 dataset Setting 2, we also achieved recall rates that exceeded 96% in four categories, with the remaining two



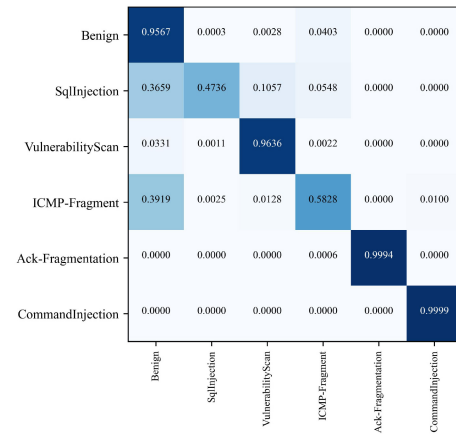
(a) CICIDS-2017-Setting1.



(b) CICIDS-2017-Setting2.



(c) CICIoT-2023-Setting1.



(d) CICIoT-2023-Setting2.

Fig. 4. Normalized Confusion Matrix of the Our Method.

at 47% and 58%. Both experiments consistently showcased the superior performance of our proposed method on this dataset. Additionally, we will validate the effectiveness of payload features in network intrusion detection, as elaborated in Section V-D.

D. Evaluation of Payload Features

We evaluated the enhancement of payload features in our proposed DM-IDS using the CICIDS-2017 dataset and CICIoT-2023 dataset in two settings: single-modal with flow features (Flow Stream), single-modal with payload features (Payload Stream) and dual-modal with bilinear fusion (Bilinear Fusion). The model parameters remained consistent for all three methods. Table VI presents the experimental results, with metrics highlighted in red and bold indicating the improvement achieved by the bilinear fusion method over the single-modal flow feature method.

In CICIoT-2023-Setting 1, our bilinear fusion method showed performance improvements in all five label categories compared to the single-modal flow feature method. Particularly notable was the significant enhancement in detecting Mirai-Greeth-Flood attacks, with the recall rate increasing

from 3.86% to 72.08%, a remarkable improvement of nearly 70 percentage points. Furthermore, substantial improvements in precision and F1 score were observed. In CICIoT-2023-Setting 2, similar enhancements were observed across the five label categories, with recall rates for SqlInjection and DDoS-ICMP-Fragment attacks increasing by 4 to 6 percentage points. Furthermore, compared to the single-modal payload feature method, the bilinear fusion method demonstrated superior performance.

In CICIDS-2017-Setting1, our proposed bilinear fusion method achieved recall, precision, and F1 scores of over 99% across all categories. In contrast, the performance of the model in the *Flow Stream* and *Payload Stream* experiments was not as good as bilinear fusion method, with recall ranging from 90% to 98%, and precision and F1 scores even slightly lower. Similarly, this trend was observed in CICIDS-2017-Setting2.

From the above results, it is evident that our proposed multimodal fusion method outperforms flow-based methods. This can be attributed to the incorporation of semantic payload information into our model, which enhances our detection performance. Further discussion on the choice of fusion methods will be presented in Section V-E.

TABLE VI
COMPARISON BETWEEN SINGLE-MODAL AND DUAL-MODAL METHOD

Datasets	Label	Flow Stream			Payload Stream			Bilinear Fusion		
		Re.	Pr.	F1	Re.	Pr.	F1	Re.	Pr.	F1
CICIoT-2023-Setting1	Benign	97.28%	46.68%	63.09%	95.08%	67.73%	79.11%	99.00%	68.80%	81.18%
	DDoS-Ack-Fragment	99.94%	99.99%	99.97%	99.99%	99.99%	99.99%	99.94%	99.99%	99.97%
	DDoS-HttP-Flood	98.11%	99.10%	98.60%	97.97%	99.35%	98.66%	99.36%	99.99%	99.68%
	DDoS-ICMP-Fragment	49.72%	56.72%	52.99%	41.00%	48.94%	44.62%	52.53%	66.42%	58.66%
	Mirai-Greeth-Flood	3.86%	62.05%	7.27%	57.81%	94.25%	71.66%	72.03%	92.28%	80.90%
	VulnerabilityScan	95.92%	97.10%	96.51%	93.78%	80.92%	86.88%	96.50%	96.82%	96.66%
CICIoT-2023-Setting2	Benign	94.61%	63.96%	76.32%	91.11%	66.90%	77.15%	95.67%	66.68%	78.59%
	SqlInjection	43.64%	99.11%	60.60%	45.01%	96.23%	61.33%	47.36%	94.53%	63.10%
	VulnerabilityScan	95.92%	97.43%	96.67%	90.58%	98.02%	94.15%	96.36%	96.93%	96.64%
	DDoS-ICMP-Fragment	52.81%	88.30%	66.09%	31.39%	74.93%	44.24%	58.17%	91.96%	71.26%
	DDoS-Ack-Fragment	99.94%	99.99%	99.97%	99.99%	99.99%	99.99%	99.94%	99.99%	99.97%
	CommandInjection	99.99%	97.85%	98.91%	99.77%	69.06%	81.62%	99.99%	98.81%	99.40%
CICIDS-2017-Setting1	Benign	93.03%	94.73%	93.87%	95.86%	93.70%	94.77%	99.73%	99.92%	99.82%
	FTP-Patator	98.15%	99.53%	98.84%	99.99%	96.80%	98.37%	99.99%	99.99%	99.99%
	SSH-Patator	98.67%	89.71%	93.98%	99.99%	99.51%	99.75%	99.99%	99.99%	99.99%
	Dos-Slowloris	97.10%	95.63%	96.36%	62.10%	93.99%	74.79%	99.90%	99.60%	99.75%
	Dos-SlowHttptest	98.38%	96.89%	97.63%	56.61%	61.67%	59.03%	99.63%	99.91%	99.77%
	Dos-Hulk	90.19%	95.49%	92.76%	70.55%	49.62%	58.26%	99.81%	99.62%	99.72%
	DDoS	98.50%	99.36%	98.93%	99.92%	99.02%	98.47%	99.99%	99.99%	99.99%
	Bot	99.99%	95.86%	97.89%	77.01%	88.71%	82.45%	99.99%	99.86%	99.93%
CICIDS-2017-Setting2	Benign	93.03%	97.32%	95.12%	92.98%	94.84%	93.90%	99.56%	99.91%	99.73%
	FTP-Patator	98.82%	97.55%	98.18%	99.07%	95.29%	97.14%	99.89%	99.89%	99.89%
	SSH-Patator	98.45%	87.85%	92.85%	99.99%	93.66%	96.73%	99.99%	99.99%	99.99%
	Dos-SlowHttptest	99.63%	98.43%	99.02%	88.55%	86.89%	87.71%	99.91%	99.99%	99.95%
	Dos-GoldenEye	99.01%	98.53%	98.77%	60.19%	67.01%	63.41%	99.99%	99.99%	99.99%
	Brute-Force	91.86%	97.53%	94.61%	78.77%	76.63%	77.69%	99.99%	97.59%	98.78%
	PortScan	12.50%	99.99%	22.22%	0.000%	0.000%	0.000%	98.18%	98.18%	98.18%
	Bot	99.99%	94.89%	97.38%	82.51%	80.86%	81.68%	99.81%	99.81%	99.81%

E. Evaluation of Different Fusion Method

To compare the impact of different fusion methods on model performance, we conducted experiments on the CICIDS-2017 dataset using three distinct fusion methods: simple concatenation, attention-based fusion, and bilinear fusion. Throughout the subsequent discussions, we abbreviate them as “*concat fusion*”, “*attn fusion*”, and “*bilinear fusion*”, respectively. The model architecture and parameters remained consistent across all three experiments, except for the fusion method. The outcomes are outlined in Table VII, where the experimental data in the leading position are highlighted in bold.

Analyzing the data presented in Table VII reveals that, in the CICIDS-2017-Setting1 dataset, we show leadership in 6 of the 8 distinct categories while achieving parity in the remaining 2 categories. Likewise, on the CICIDS-2017-Setting2 dataset, we lead in 7 categories. Based on experimental data, it is shown that the bilinear fusion method emerges as the most effective in our network intrusion detection application.

F. Evaluation of Zero-Day Detection

To evaluate our proposed DM-IDS method for detecting unknown attacks, we conducted four experiments on the CICIoT-2023 dataset. First, we select 9 types of attack traffic not present in the CICIoT-2023 dataset Setting 1 and Setting 2 as our unknown attacks, as detailed in Fig. 5. When selecting the zero-day attack samples, we followed two key principles:

- *The attacks exhibit some similarities to known attacks in terms of mechanism but are not exactly the same.*

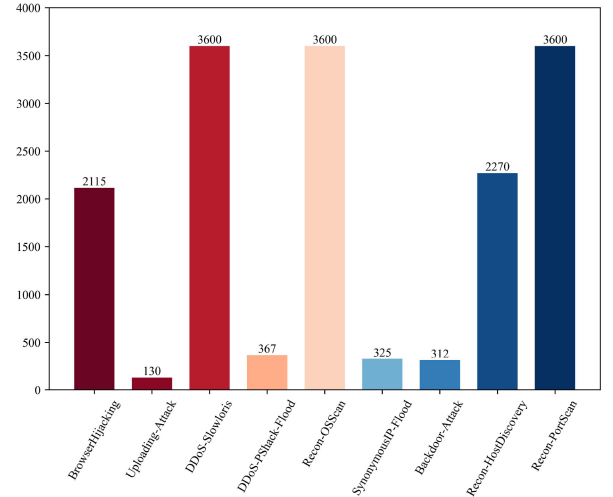


Fig. 5. Traffic Details of Unknown Datasets.

For example, DDoS-Slowloris, DDoS-PSHACK-Flood, and DDoS-SynonymousIP all fall under the DDoS category, which is also present in the known attack set. However, these attacks differ in their specific mechanisms and do not belong to the same type of DDoS attack.

- *The attacks have entirely different mechanisms compared to known attacks.* For example, Recon-OSScan, Recon-HistoryDiscovery, and Recon-Portscan are all reconnaissance attacks, but these types are not present in either Setting 1 or Setting 2 of the CICIoT-2023 dataset.

TABLE VII
COMPARING TO DIFFERENT FUSION METHOD

Datasets	Label	Concat Fusion			Attn Fusion			Bilinear Fusion		
		Re.	Pr.	F1	Re.	Pr.	F1	Re.	Pr.	F1
CICIDS-2017-Setting1	Benign	99.10%	99.76%	99.43%	92.11%	96.28%	94.15%	99.73%	99.92%	99.82%
	FTP-Patator	99.88%	99.65%	99.77%	99.88%	99.19%	99.54%	99.99%	99.99%	99.99%
	SSH-Patator	99.66%	99.99%	99.83%	98.74%	86.69%	92.32%	99.99%	99.99%	99.99%
	Dos-Slowloris	99.81%	99.81%	99.81%	97.53%	99.04%	98.28%	99.90%	99.60%	99.75%
	Dos-SlowHttptest	99.53%	99.72%	99.63%	99.12%	96.83%	97.96%	99.63%	99.91%	99.77%
	Dos-Hulk	99.71%	99.42%	99.56%	93.20%	95.51%	94.34%	99.81%	99.62%	99.72%
	DDoS	99.92%	99.92%	99.92%	97.64%	99.52%	98.57%	99.99%	99.99%	99.99%
	Bot	99.99%	99.27%	99.63%	99.95%	98.10%	99.02%	99.99%	99.86%	99.93%
CICIDS-2017-Setting2	Benign	98.02%	99.54%	98.77%	93.53%	97.34%	95.40%	99.56%	99.91%	99.73%
	FTP-Patator	99.88%	99.41%	99.64%	99.45%	97.72%	98.58%	99.89%	99.89%	99.89%
	SSH-Patator	99.99%	99.52%	99.76%	98.32%	88.91%	93.28%	99.99%	99.99%	99.99%
	Dos-SlowHttptest	99.91%	99.82%	99.86%	99.34%	98.40%	98.87%	99.91%	99.99%	99.95%
	Dos-GoldenEye	99.55%	96.93%	98.22%	99.13%	98.27%	98.70%	99.99%	99.99%	99.99%
	Brute-Force	94.05%	99.37%	96.64%	91.28%	98.74%	94.86%	99.99%	97.59%	98.78%
	PortScan	94.55%	99.99%	97.20%	32.08%	85.00%	46.58%	98.18%	98.18%	98.18%
	Bot	99.99%	98.54%	99.27%	99.86%	97.96%	98.90%	99.81%	99.81%	99.81%

TABLE VIII
EVALUATION OF ZERO-DAY DETECTION (BILINEAR FUSION)

Label	# Setting1			# Setting2		
	Re.	Pr.	F1	Re.	Pr.	F1
0	95.13%	94.91%	95.02%	87.42%	94.67%	90.90%
1	40.00%	32.50%	35.86%	88.46%	52.51%	69.50%
2	83.78%	96.54%	89.71%	57.03%	95.18%	71.32%
3	85.83%	74.47%	79.75%	35.15%	55.36%	43.00%
4	90.03%	96.78%	93.28%	33.28%	92.01%	48.88%
5	95.38%	79.69%	86.83%	54.46%	60.00%	57.10%
6	38.14%	60.10%	46.67%	91.03%	70.65%	79.55%
7	56.12%	94.16%	70.33%	46.26%	89.90%	61.08%
8	87.19%	97.55%	92.08%	22.83%	87.45%	36.21%

* 0: BrowserHijacking; 1: Uploading-Attack; 2: DDoS-Slowloris; 3: DDoS-PShack-Flood; 4: Recon-OSScan; 5: DDoS-SynonymousIP-Flood; 6: Backdoor; 7: Recon-HostDiscovery; 8: Recon-PortScan

TABLE IX
EVALUATION OF ZERO-DAY DETECTION (FLOW STREAM)

Label	# Setting1			# Setting2		
	Re.	Pr.	F1	Re.	Pr.	F1
0	92.86%	92.25%	92.55%	84.92%	93.54%	89.02%
1	19.23%	13.16%	15.62%	19.23%	16.78%	17.92%
2	59.19%	92.81%	72.29%	57.22%	94.32%	71.23%
3	36.24%	44.63%	40.00%	34.60%	50.60%	41.10%
4	62.11%	93.13%	74.52%	7.47%	68.45%	13.47%
5	56.62%	25.56%	35.22%	50.15%	49.85%	50.00%
6	31.73%	15.59%	20.91%	26.60%	33.60%	29.70%
7	55.68%	70.22%	62.11%	25.33%	77.81%	38.22%
8	66.75%	81.76%	73.50%	10.19%	69.11%	17.77%

* 0: BrowserHijacking; 1: Uploading-Attack; 2: DDoS-Slowloris; 3: DDoS-PShack-Flood; 4: Recon-OSScan; 5: DDoS-SynonymousIP-Flood; 6: Backdoor; 7: Recon-HostDiscovery; 8: Recon-PortScan

By using these two distinct approaches, our objective is to comprehensively evaluate the performance of our model in detecting unknown attacks. Furthermore, we consider them correctly predicted if they were identified as attacks during the prediction. The experimental results are shown in Table VIII and Table IX, where we labeled the two models trained in the CICIoT-2023 dataset as Setting 1 and Setting 2, respectively.

Table VIII shows that DM-IDS has better detection capability for unknown attacks in Setting 1, the recall rates for the seven unknown attack types are greater than 84%, with three exceeding 90%. In Setting 2, DM-IDS performs slightly worse for zero-day attacks, with recall rates above 87% for three attack types, around 50% for four others, and below 50% for the remaining three. Our method excels not only in detecting known attacks, but also in detecting zero-day attacks, particularly in Setting 1. This is due to our model capturing more potential features of attack types, such as flow and payload features, which contribute significantly to our performance in attack detection.

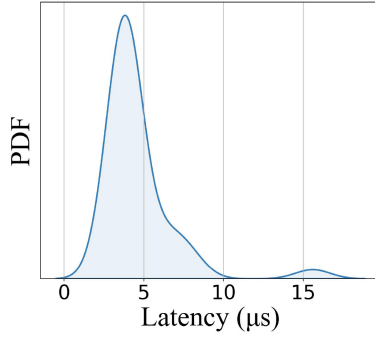
To demonstrate the improved performance of DM-IDS over the unimodal model in detecting unknown attacks, we conducted two additional experiments under the Flow

Stream model. The results are presented in Table IX. For the model trained on the Setting 1 dataset, the detection results for unknown attacks show significant improvements in recall rate for attacks such as Uploading-Attack, DDoS-PShack-Flood, DDoS-SynonymousIP-Flood, Backdoor, Recon-HostDiscovery, and Recon-PortScan. In addition, notable improvements were observed in precision and F1 score. Similarly, for the model trained on the Setting 2 dataset, the recall rate for unknown attacks, including Uploading-Attack, Recon-OSScan, DDoS-SynonymousIP-Flood, Backdoor, Recon-HostDiscovery, and Recon-PortScan, exhibited substantial increases. The enhancements in precision and F1 score metrics mirrored those observed in the first set of experiments.

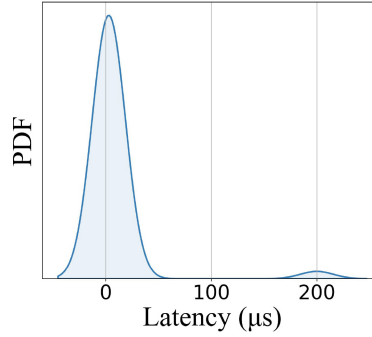
These two experiments further substantiate that DM-IDS achieves considerable improvements compared to the model trained on the flow modal. By incorporating packet modality and using a binary vectorization approach for packet payloads, rather than treating them as natural language, the model demonstrates enhanced capability in capturing attack features, which is a key factor behind its superior performance.

TABLE X
COMPARING TO STATE OF THE ART

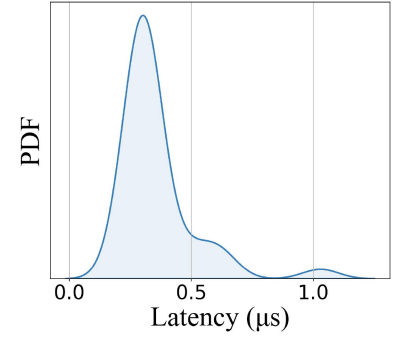
Datasets	Label	RF-Vote-Based			MTH-IDS			CVAE-EVT			DM-IDS		
		Re.	Pr.	F1	Re.	Pr.	F1	Re.	Pr.	F1	Re.	Pr.	F1
CICIDS-2017 Setting1	Benign	99.07%	98.22%	98.64%	99.78%	98.60%	99.19%	94.78%	98.90%	96.79%	99.73%	99.92%	99.82%
	FTP-Patator	99.99%	99.99%	99.99%	99.68%	99.68%	99.68%	99.46%	99.40%	99.43%	99.99%	99.99%	99.99%
	SSH-Patator	97.29%	98.55%	97.92%	98.01%	99.79%	98.89%	99.92%	95.29%	97.55%	99.99%	99.99%	99.99%
	Slowloris	99.13%	99.71%	99.42%	95.10%	99.43%	97.22%	99.77%	98.57%	99.17%	99.90%	99.60%	99.75%
	SlowHttpptest	99.61%	99.99%	99.81%	99.70%	94.52%	97.04%	99.36%	99.92%	99.64%	99.63%	99.91%	99.77%
	Hulk	98.36%	99.22%	98.79%	93.28%	99.72%	96.39%	99.87%	96.52%	98.16%	99.81%	99.62%	99.72%
	DDoS	99.68%	99.99%	99.84%	99.72%	99.48%	99.60%	98.44%	99.93%	99.18%	99.99%	99.99%	99.99%
	Bot	99.99%	98.70%	99.35%	98.53%	93.06%	95.71%	99.00%	94.30%	96.59%	99.99%	99.86%	99.93%
	Macro Avg	99.14%	99.30%	99.22%	97.98%	98.04%	97.97%	98.82%	97.85%	98.31%	99.88%	99.86%	99.87%
	Weighted Avg	99.18%	99.18%	99.18%	98.82%	98.84%	98.81%	98.29%	98.33%	98.29%	99.87%	99.87%	99.87%
CICIDS-2017 Setting2	Benign	98.00%	99.31%	98.65%	99.25%	98.90%	99.07%	95.70%	99.16%	97.40%	99.56%	99.91%	99.73%
	FTP-Patator	99.88%	99.99%	99.94%	99.68%	98.98%	99.33%	99.68%	99.75%	99.72%	99.89%	99.89%	99.89%
	SSH-Patator	98.51%	95.36%	96.91%	98.85%	99.68%	99.27%	99.59%	97.02%	98.29%	99.99%	99.99%	99.99%
	SlowHttpptest	99.99%	99.82%	99.91%	99.90%	97.06%	98.46%	99.24%	99.77%	99.50%	99.91%	99.99%	99.95%
	GoldenEye	99.55%	99.99%	99.77%	86.68%	99.72%	92.75%	99.99%	98.49%	99.24%	99.99%	99.99%	99.99%
	Brute-Force	92.00%	98.77%	95.27%	92.21%	96.38%	94.25%	94.57%	87.29%	90.78%	99.99%	97.59%	98.78%
	PortScan	92.98%	98.15%	95.50%	80.73%	74.58%	77.53%	99.76%	99.99%	99.88%	98.18%	98.18%	98.18%
	Bot	99.99%	96.80%	98.37%	98.53%	95.71%	97.10%	99.02%	90.81%	94.74%	99.95%	99.72%	99.84%
	Macro Avg	97.62%	98.53%	98.04%	94.48%	95.13%	94.72%	98.45%	96.54%	97.44%	99.69%	99.41%	99.55%
	Weighted Avg	98.77%	98.79%	98.77%	98.47%	98.50%	98.46%	98.15%	98.24%	98.17%	99.81%	99.81%	99.81%



(a) Latency of FNet.



(b) Latency of PNet.



(c) Latency of Fusion.

Fig. 6. Performance Test in the CICIDS-2017 Dataset.

G. Comparing to State-of-the-Art

Our proposed approach is compared with three state-of-the-art methods, including RF-Vote-Based [24], MTH-IDS [17] and CVAE-EVT [16]. For a fair comparison, the experimental results regarding these state-of-the-art models were obtained by combining our dataset with the deployment code provided by the respective authors. The comparative results for all models are presented in Table X (we have highlighted the significantly leading metrics in bold).

As shown in Table X, considering all three performance metrics, our proposed DM-IDS model clearly outperforms all state-of-the-art methods. RF-Vote-Based method demonstrated strong performance in the CICIDS-2017 data set, achieving a recall rate that exceeded 98% in most categories. However, DM-IDS outperformed RF-Vote-Based in recall rate for specific attack types such as Benign, SSH-Patator, and Bot, as well as in precision and F1 score metrics. Compared with CVAE-EVT, DM-IDS demonstrates highly competitive experimental results, achieving comprehensive leadership across nearly all categories. Although MTH-IDS exhibits commendable performance in the two settings of the CICIDS-2017

dataset, our proposed method, DM-IDS, notably excels in categories such as FTP-Patator, DoS-Slowloris, DDoS, Bot, Brute-Force, Portscan, demonstrating significant performance improvements. Furthermore, we maintain nearly identical leading results in various other categories. This indicates that DM-IDS can better capture payload feature information, enhancing the model's ability to conduct significant detection with increased sensitivity. We will delve further into the discussion of the experiments in the **Discussion** section.

H. Performance Test

Fig. 6 presents the evaluation of the detection latency of DM-IDS in three critical modules - FNet, PNet, and Fusion—using the CICIDS-2017 dataset. Fig. 6 (a) illustrates the latency density distribution for FNet, with the majority of latency concentrated within 0–10 microseconds and a maximum latency of approximately 15 microseconds, although the density in this upper range is relatively low. Fig. 6 (b) shows the latency density distribution for PNet, primarily within 0 to 50 microseconds, with a maximum latency of around 200 microseconds, which also demonstrates a low density.

Finally, Fig. 6 (c) shows the latency density distribution for the Fusion module, which exhibits extremely low latency, predominantly below 1 microsecond.

As shown in the latency density distributions of the three different modules in Fig. 6, the introduction of PNet results in an increase in detection latency compared to FNet. However, this latency remains relatively low, primarily under 50 microseconds, with the average latency skewed closer to 0. We attribute this increase to the larger feature dimensions in the payload modality, which leads to higher computational complexity. In general, the total detection latency of DM-IDS is expected to remain within 60 microseconds. By incorporating a parallel mechanism, where PNet and FNet perform detection simultaneously before merging modalities, the overall detection latency could be further reduced. Such low latency is well suited to achieve real-time performance in practical deployment scenarios.

VI. DISCUSSION

Enhancement in Recall Performance: In contrast to current state-of-the-art (SOTA) methods, we have achieved a notable improvement in the recall rates for detecting Bot, Brute-Force, and Dos-Slowloris attacks. Bot attacks involve malicious activities executed on a target system through automated programs, commonly known as bots. The payload may include malicious operations, data, or instructions executed by the automated program controlled by the attacker during the attack. Conversely, Brute-Force attacks employ trial-and-error methods in which attackers seek unauthorized access or information by testing all feasible password, key, or credential combinations. The payload content typically encompasses the targets that attackers attempt to decipher, often passwords, keys, or other credentials. It is evident that the payload content of these attacks exhibits distinctive features. Our approach is designed to extract the payload features of the attacks and integrate them with flow modal features to more effectively fulfill the detection task.

Fusion Method Selection: Our investigation into different fusion methods revealed varied performance outcomes. Initially, the concatenation fusion method combines two modal feature vectors by adding them row-wise or column-wise. However, this method often falls short in capturing the correlations between the two modal features. Subsequently, the attention-based fusion method acknowledges the correlations between the two modal features. Nevertheless, its implementation suggests a potential bias towards favoring one modality after fusion, potentially diminishing the influence of the other. Finally, the bilinear fusion method that we adopted shares similarities with the attention-based fusion method in implementation, but differs in key aspects. Notably, it avoids biased selection towards either modality, a characteristic evident in our experimental results.

VII. FUTURE WORK

In recent years, significant strides have been made in deep learning research for network intrusion detection, and we have invested substantial time and effort in this domain, resulting

in meaningful results. However, there are still several pressing challenges in IDS research that remain to be addressed and will form an important part of our future work.

- We acknowledge that the ultimate goal of intrusion detection is to thwart attacks, which first requires accurate identification and interpretation of malicious behavior. Although traditional methods often perform well in this regard, deep learning-based approaches have yet to fully achieve this objective. In addition, encrypted traffic, particularly SSL-encrypted data, remains a major challenge for intrusion detection.
- Extracting more effective features is in fact a key component of IDS research, such as evaluating the contribution of features and eliminating redundant features. These steps can play a critical role in improving detection performance as well as improving real-time efficiency.
- Model generalization is a crucial aspect of classification tasks, and this is equally important in IDS research. Our work is carried out under a static assumption, where the distribution of network traffic features remains unchanged over a certain period. Although this assumption holds in the short term, concept drift and other phenomena may occur over time, necessitating continuous model updates to maintain strong generalization performance [14]. Addressing this challenge represents an important direction for future IDS research.

VIII. CONCLUSION

In this study, we introduce a network intrusion detection method grounded in bilinear fusion, with the primary objective of improving the performance of intrusion detection systems. Our initial step involved the autonomous development of a specialized feature extractor named Beeman, proficient in extracting features from both flow and payload modalities. Subsequently, capitalized on the dual utilization of these modalities, we crafted a *Flow Net* incorporating attention mechanisms and a *Payload Net* structured around multiple parallel convolutional blocks. The culmination of our approach involved bilinear fusion and subsequent classification applied to the encoded feature space. Throughout our comprehensive experiments, we systematically assessed the impact of various fusion methods on model performance, concluding that bilinear fusion stands out as the optimal solution. Furthermore, we conducted rigorous comparisons with several state-of-the-art approaches, thus accentuating the advancements and superiorities inherent in our proposed method within the domain of network intrusion detection.

REFERENCES

- [1] W. A. Al-Khater, S. Al-Maadeed, A. A. Ahmed, A. S. Sadiq, and M. K. Khan, "Comprehensive review of cybercrime detection techniques," *IEEE Access*, vol. 8, pp. 137293–137311, 2020.
- [2] M. Frustaci, P. Pace, G. Aloï, and G. Fortino, "Evaluating critical security issues of the IoT world: Present and future challenges," *IEEE Internet Things J.*, vol. 5, no. 4, pp. 2483–2495, Aug. 2018.
- [3] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Analyzing leakage of personally identifiable information in language models," 2023, *arXiv:2302.00539*.

- [4] Y. Zou, J. Zhu, X. Wang, and L. Hanzo, "A survey on wireless security: Technical challenges, recent advances, and future trends," *Proc. IEEE*, vol. 104, no. 9, pp. 1727–1765, Sep. 2016.
- [5] W. Li, W. Meng, and L. F. Kwok, "Surveying trust-based collaborative intrusion detection: State-of-the-art, challenges and future directions," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 1, pp. 280–305, 1st Quart., 2021.
- [6] F. Wang, Q. Shan, F. Teng, Z. He, Y. Xiao, and Z. Wang, "Distributed secondary control strategy against bounded FDI attacks for microgrid with layered communication network," *Front. Energy Res.*, vol. 10, Jun. 2022, Art. no. 914132.
- [7] V. Kumar, D. Sinha, A. K. Das, S. C. Pandey, and R. T. Goswami, "An integrated rule based intrusion detection system: Analysis on UNSW-NB15 data set and the real time online dataset," *Clust. Comput.*, vol. 23, pp. 1397–1418, Jun. 2020.
- [8] R. Mitchell and R. Chen, "Behavior-rule based intrusion detection systems for safety critical smart grid applications," *IEEE Trans. Smart Grid*, vol. 4, no. 3, pp. 1254–1263, Sep. 2013.
- [9] B. Dong and X. Wang, "Comparison deep learning method to traditional methods using for network intrusion detection," in *Proc. 8th IEEE Int. Conf. Commun. Softw. Netw. (ICCSN)*, 2016, pp. 581–585.
- [10] P. Mishra, V. Varadharajan, U. Tupakula, and E. S. Pilli, "A detailed investigation and analysis of using machine learning techniques for intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 686–728, 1st Quart., 2018.
- [11] Z. Hang, Y. Lu, Y. Wang, and Y. Xie, "Flow-MAE: Leveraging masked AutoEncoder for accurate, efficient and robust malicious traffic classification," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, 2023, pp. 297–314.
- [12] I. J. King, X. Shu, J. Jang, K. Eykholt, T. Lee, and H. H. Huang, "EdgeTorrent: Real-time temporal graph representations for intrusion detection," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, 2023, pp. 77–91.
- [13] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," 2023, *arXiv:2301.13686*.
- [14] C. Zha et al., "A-NIDS: Adaptive network intrusion detection system based on clustering and stacked CTGAN," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3204–3219, 2025.
- [15] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2021, pp. 3366–3383.
- [16] J. Yang, X. Chen, S. Chen, X. Jiang, and X. Tan, "Conditional variational auto-encoder and extreme value theory aided two-stage learning approach for intelligent fine-grained known/unknown intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3538–3553, 2021.
- [17] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.
- [18] C. Zha et al., "SKT-IDS: Unknown attack detection method based on sigmoid kernel transformation and encoder-decoder architecture," *Comput. Security*, vol. 146, Nov. 2024, Art. no. 104056.
- [19] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2021, pp. 3431–3446.
- [20] G. Marín, P. Casas, and G. Capdehourat, "Deep in the dark-deep learning-based malware traffic detection without expert knowledge," in *Proc. IEEE Security Privacy Workshops (SPW)*, 2019, pp. 36–42.
- [21] Y. Zhang, X. Chen, L. Jin, X. Wang, and D. Guo, "Network intrusion detection: Based on deep hierarchical network and original flow data," *IEEE Access*, vol. 7, pp. 37004–37016, 2019.
- [22] B. Wang, Y. Su, M. Zhang, and J. Nie, "A deep hierarchical network for packet-level malicious traffic detection," *IEEE Access*, vol. 8, pp. 201728–201740, 2020.
- [23] Y. A. Farrukh, S. Wali, I. Khan, and N. D. Bastian, "XG-NID: Dual-modality network intrusion detection using a heterogeneous graph neural network and large language model," 2024, *arXiv:2408.16021*.
- [24] A. Kiflay, A. Tsokanos, M. Fazlali, and R. Kirner, "Network intrusion detection leveraging multimodal features," *Array*, vol. 22, Jul. 2024, Art. no. 100349.
- [25] E. Min, J. Long, Q. Liu, J. Cui, and W. Chen, "TR-IDS: Anomaly-based intrusion detection through text-convolutional neural network and random forest," *Security Commun. Netw.*, vol. 2018, no. 1, 2018, Art. no. 4943509.
- [26] A. Premkumar, M. Schneider, C. Spivey, J. Pavlik, and N. D. Bastian, "Graph representation learning for context-aware network intrusion detection," in *Proc. Artif. Intell. Mach. Learn. Multi-Domain Oper. Appl. V*, 2023, pp. 82–92.
- [27] W. Wang, X. Du, D. Shan, R. Qin, and N. Wang, "Cloud intrusion detection method based on stacked contractive auto-encoder and support vector machine," *IEEE Trans. cloud Comput.*, vol. 10, no. 3, pp. 1634–1646, Jul.–Sep. 2022.
- [28] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, vol. 1, 2018, pp. 108–116.
- [29] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.
- [30] Z. Wang et al., "Themis: Ambiguity-aware network intrusion detection based on symbolic model comparison," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2021, pp. 3384–3399.
- [31] Y. Wu et al., "Paradise: Real-time, generalized, and distributed provenance-based intrusion detection," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1624–1640, Mar./Apr. 2023.
- [32] Y. Wu, L. Nie, S. Wang, Z. Ning, and S. Li, "Intelligent intrusion detection for Internet of Things security: A deep convolutional generative adversarial network-enabled approach," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3094–3106, Feb. 2023.
- [33] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 1, pp. 41–50, Feb. 2018.
- [34] M. Verkerken et al., "A novel multi-stage approach for hierarchical intrusion detection," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3915–3929, Sep. 2023.
- [35] G. Duan, H. Lv, H. Wang, and G. Feng, "Application of a dynamic line graph neural network for intrusion detection with semisupervised learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 699–714, 2022.
- [36] M. Shafiq, Z. Tian, A. K. Bashir, X. Du, and M. Guizani, "CorrAUC: A malicious bot-IoT traffic detection method in IoT network using machine-learning techniques," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3242–3254, Mar. 2021.
- [37] H. He et al., "Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection," in *Proc. 33rd USENIX Security Symp. (USENIX Security)*, 2024, pp. 1759–1776.
- [38] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 3320–3328.
- [39] A. S. Razavian, H. Azizpour, J. Sullivan, and S. Carlsson, "CNN features off-the-shelf: An astounding baseline for recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2014, pp. 806–813.
- [40] D. Clark, "The design philosophy of the DARPA Internet protocols," in *Proc. Commun. Archit. Protocols Symp.*, 1988, pp. 106–114.
- [41] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–15.
- [42] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [43] C. Szegedy et al., "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1–9.
- [44] N. Huang, Y. Yang, D. Zhang, Q. Zhang, and J. Han, "Employing bilinear fusion and saliency prior information for RGB-D salient object detection," *IEEE Trans. Multimedia*, vol. 24, pp. 1651–1664, 2022.
- [45] C. Zhang, Z. Yang, X. He, and L. Deng, "Multimodal intelligence: Representation learning, information fusion, and applications," *IEEE J. Sel. Topics Signal Process.*, vol. 14, no. 3, pp. 478–493, Mar. 2020.
- [46] J. Chen and C. M. Ho, "MM-ViT: Multi-modal video transformer for compressed video action recognition," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2022, pp. 1910–1921.
- [47] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Security Privacy Workshops (SPW)*, 2021, pp. 7–12.
- [48] L. D'Hooge, M. Verkerken, B. Volckaert, T. Wauters, and F. De Turck, "Establishing the contaminating effect of metadata feature inclusion in machine-learned network intrusion detection models," in *Proc. Int. Conf. Detect. Intrusions Malware, Vulnerability Assessment*, 2022, pp. 23–41.



Chao Zha is currently pursuing the Ph.D. degree in computer science and technology with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, under the supervision of Prof. R. Zhang. His research interests include artificial intelligence for cyberspace security, malware detection, and intrusion detection.



Yinjie Zhang received the M.S. degree in communication engineering from Harbin Engineering University. He is currently employed with the Intelligent Computing Infrastructure Innovation Center, Zhejiang Lab. His research interests include natural language processing, adversarial samples, and large language models.



Zhiyu Wang received the B.S. and Ph.D. degrees in automatic control from the College of Control Science and Engineering, Zhejiang University, Hangzhou, China, in 2016 and 2021, respectively. He is currently with the Zhejiang Laboratory, Hangzhou. His research interests include industrial control systems and their security.



Sainan Shi received the B.S. degree from Central South University in 2022. She is currently pursuing the M.S. degree in electronic information with the Institute of Computing Technology, Chinese Academy of Sciences. Her research interests include artificial intelligence security.



Yifei Fan received the M.S. degree in mathematica applicata from Columbia University. He is a Research Engineer with the Intelligent Computing Infrastructure Innovation Center, Zhejiang Lab, working on artificial intelligence applications and artificial intelligence security.



Bing Bai received the M.S. degree in computer science and technology from the National University of Defense Technology. He is currently working as an Associate Investigator with the Intelligent Computing Infrastructure Innovation Center, Zhejiang Lab. His research interests include communication networks and security.



Ruyun Zhang received the Ph.D. degree in communications and information systems in 2011. Having been a Researcher and a Ph.D. Supervisor for years, he is currently working as the Deputy Director of the Intelligent Computing Infrastructure Innovation Center, Zhejiang Lab. His research interests include communication networks and security.